

Port POC Plan

Create Repository and Configure Pipeline
(Phases, Epics & User Stories)

Based on the customer UML sequence diagram
Orchestration: Port Workflows · IaC: Terraform via GitHub Actions
Document language: English

1. POC goal and approach

Goal: Prove one end-to-end path a developer can run from Port, with DevOps approval and status back to the requester — not full production parity on day one.

Key design choice for the POC: Use Port’s native INPUT approval node for the Terraform plan→apply gate (matches “DevOps approves in Port” in the diagram). Keep GitHub Issues as a later optional story if the customer requires issue-based tracking.

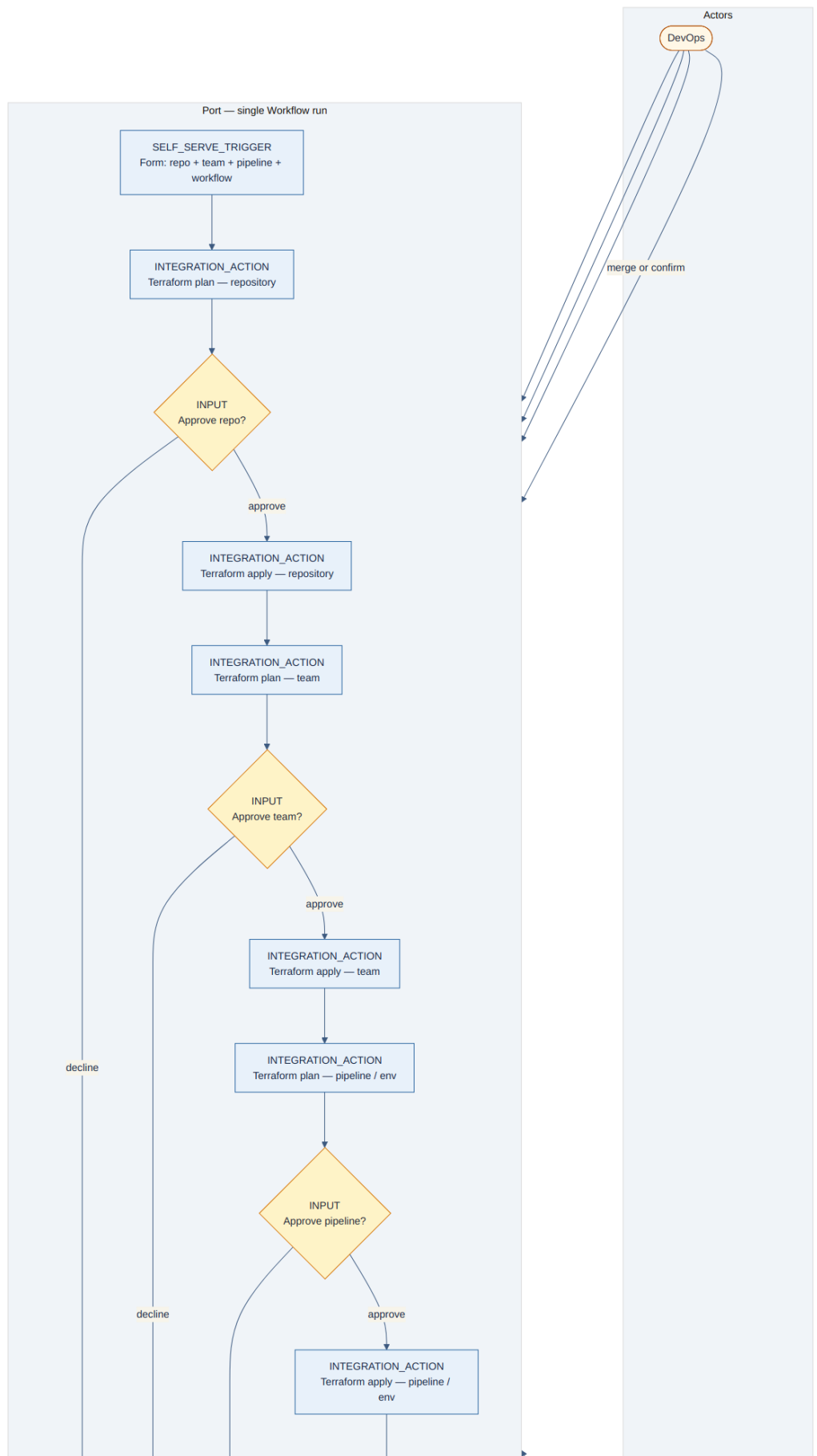
Slice strategy: Build one vertical slice first (addRepository), then reuse the same plan→approve→apply skeleton for team and env vars. Treat addWorkflow (PR merge) as a separate final slice — or keep it in the same workflow behind a PR-merge / confirmation gate (see architecture).

2. Architecture: single workflow with approvals

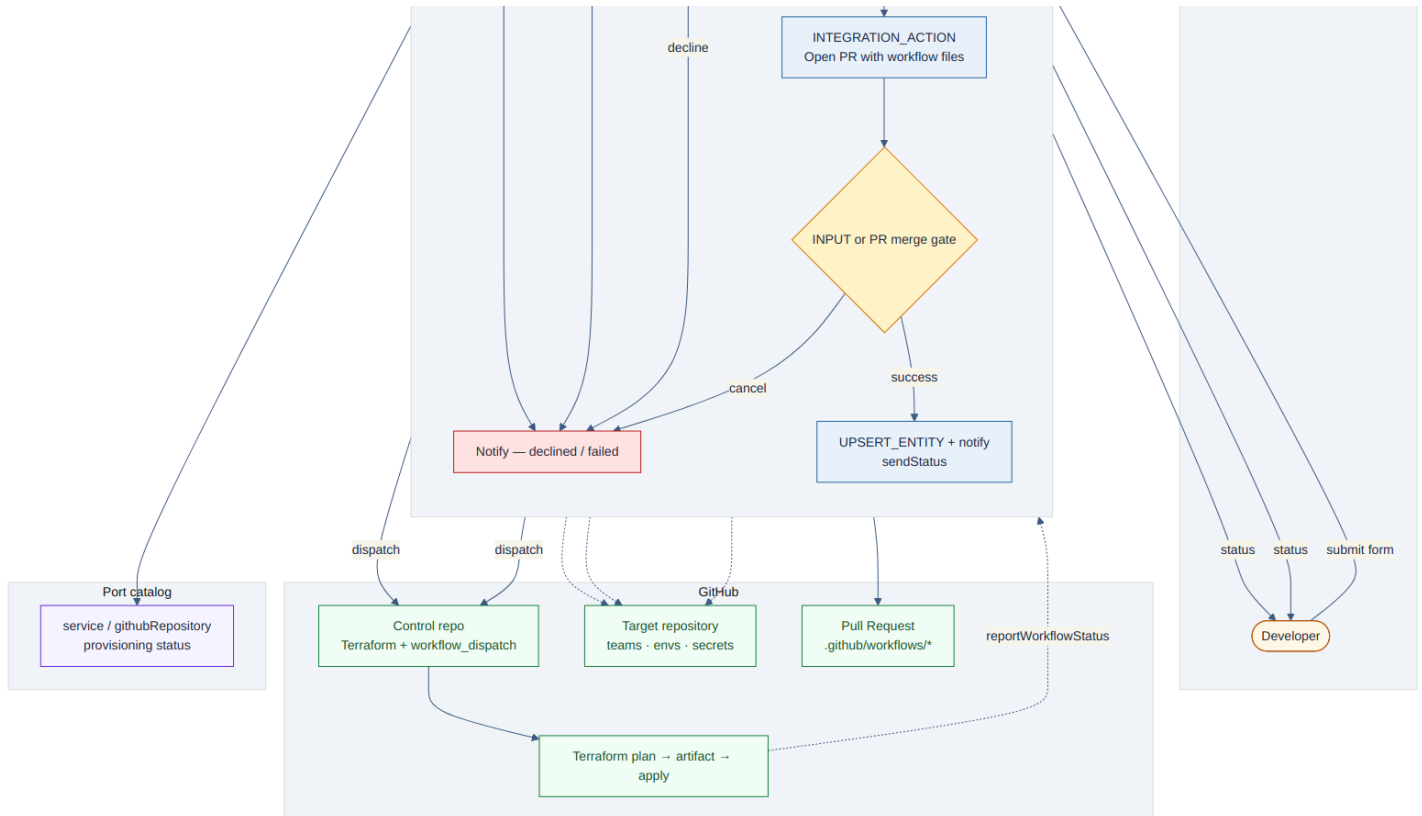
The full journey (create repository → add team → configure pipeline/env vars → add workflow) is modeled as one Port Workflow run, with INPUT approval nodes between Terraform plan/apply stages. The final phase uses a PR-merge gate (or a confirmation INPUT) instead of the plan/apply loop.

- One workflow run owns the whole journey; each diamond is a human gate.
- Phases 1–3 share the same pattern: Port dispatches GitHub Actions → Terraform plan → DevOps INPUT → apply.
- Phase 4 still sits in the same workflow, but the gate is PR merge (or an INPUT “confirm merged”).
- Decline from any approval jumps to notify/fail so provisioning stops.

Architecture diagram — single Port workflow



Architecture diagram (continued)



3. Implementation phases

Phase 0 — Foundations

1. Connect Port ↔ GitHub (Ocean / GitHub app).
2. Create a platform control repo holding Terraform modules and workflow_dispatch GitHub Actions (plan /

apply).

3. Define minimal blueprints: service (or reuse built-in), githubRepository (from integration), optional provisioningRequest for run tracking.
4. Store Port credentials as GitHub secrets; use port-labs/port-github-action to report run status.

Phase 1 — Vertical slice: Create repository

1. Self-service workflow form: repo name, visibility, team, template.
2. INTEGRATION_ACTION → GitHub Actions plan (Terraform creates repo).
3. INPUT node → DevOps approve / decline.
4. On approve → apply workflow; on success → upsert Port entity and notify user.
5. Manual test with a throwaway org / sandbox.

Phase 2 — Reuse pattern: Add team

1. Second self-service action (or chained step after repo exists).
2. Same plan→approve→apply against team membership Terraform.
3. Relate repo ↔ team in Port (\$team / relations).

Phase 3 — Pipeline and environment variables

1. Form for pipeline type and env var keys/values (secrets via GitHub Environments / Actions secrets APIs, behind Terraform or a dedicated GHA).
2. Same approval loop; upsert pipeline/env metadata on the service entity.

Phase 4 — Add workflow files (PR gate)

1. Port dispatches an action that opens a PR with .github/workflows/* (or scaffolds from a template).
2. DevOps merges in GitHub (human gate outside Port).
3. Event trigger / polling / status callback updates Port when the PR is merged.
4. Final status notification to the requester.

Phase 5 — Glue into one journey (optional for POC)

1. Either one multi-step workflow with sequential stages, or four workflows plus a thin orchestrator that chains them via entity status events.
2. Dashboard: provisioning requests, pending approvals, repository catalog.

4. Suggested build order

1. A1-A3 + B1-B4 — smallest demo matching UML Phase 1.
2. C1-C2 — proves the pattern is reusable.
3. D1-D3 — proves secrets and pipeline configuration.
4. E1-E3 — proves the different approval mechanism (PR).
5. F1-F2 — polish for stakeholder demo.

5. Epics and user stories

Epic A — Platform foundations

A1 - As a platform engineer, I connect Port to our GitHub org so workflows can dispatch Actions.

Acceptance criteria: GitHub integration healthy; can dispatch a no-op workflow_dispatch from Port and see success.

A2 - As a platform engineer, I have a control repo with Terraform + plan/apply Actions that report status to Port.

Acceptance criteria: Plan uploads artifact; apply consumes it; both PATCH Port run logs via port-github-action.

A3 - As a platform engineer, I define blueprints for service / provisioning state.

Acceptance criteria: Entity can track status (requested → planned → approved → applied / failed) and link to GitHub repo URL.

Epic B — Create repository (addRepository)

B1 - As a developer, I submit “Create repository” from Port with name, visibility, and owning team.

Acceptance criteria: Form validates inputs; run appears in Port with requester identity.

B2 - As the system, I run Terraform plan for the new repo and pause for approval.

Acceptance criteria: Plan succeeds; DevOps sees pending INPUT approval with plan summary/link; no apply yet.

B3 - As DevOps, I approve or decline the change in Port.

Acceptance criteria: Approve → apply runs; decline → run ends, user notified, no repo created.

B4 - As a developer, I get status when provisioning finishes.

Acceptance criteria: Repo exists in GitHub; Port entity updated; run status SUCCESS/FAILURE matches reality.

POC test script (B)

- Happy path
- Decline path
- Plan failure path (invalid name)

Epic C — Add team (addTeam)

C1 - As a developer, I request team access on an existing repo from Port.

Acceptance criteria: Form selects repo entity + team; only allowed teams shown.

C2 - Same plan→approve→apply loop applies team permissions via Terraform.

Acceptance criteria: After approve, GitHub team has expected permission; Port relations/\$team updated.

POC test script (C)

- Add team
- Decline
- Attempt on a repo the user does not own (permission check)

Epic D — Pipeline & env vars (addPipeline / addEnvVars)

D1 - As a developer, I choose a pipeline template and env vars for a repo.

Acceptance criteria: Form captures non-secret + secret refs; secrets never logged in Port run output.

D2 - Plan/apply configures GitHub Environment / variables via Terraform or GHA.

Acceptance criteria: Vars present in GitHub; Port shows “pipeline configured”.

D3 - DevOps approval required before secrets/vars are written.

Acceptance criteria: No mutation until approve.

POC test script (D)

- Staging environment only
- Verify secret redaction in logs

Epic E — Add workflow files (addWorkflow)

E1 - As a developer, I request CI workflow scaffolding for my repo.

Acceptance criteria: Port creates a branch + PR with workflow YAML from a template.

E2 - As DevOps, I review and merge the PR in GitHub.

Acceptance criteria: Merge is the approval gate (as in the UML); no silent push to default branch.

E3 - As a developer, I see final status in Port after merge.

Acceptance criteria: PR merged event (or polling) updates entity; user notified.

POC test script (E)

- Open PR → merge → Port SUCCESS
- Open PR → close without merge → FAILED/CANCELLED

Epic F — Journey UX & observability (thin for POC)**F1 - As a developer, I can see where my request is stuck (plan / approval / apply / PR).**

Acceptance criteria: Single run timeline or provisioningRequest entity shows stage.

F2 - As DevOps, I have a queue of pending approvals.

Acceptance criteria: Port page/widget lists open INPUT approvals.

6. Out of scope for POC

- Full GitHub Issue-based approval mirroring the UML (use Port INPUT first).
- Multi-org / multi-cloud Terraform backends.
- Automated chaining of all four phases into one click without intermediate confirmation.
- Production RBAC hardening beyond “Member can request / DevOps team can approve”.

7. UML → Port construct mapping

- User addRepositorie(form) → Workflow SELF_SERVE_TRIGGER
- Port → GithubAPI createRepo → Indirect via Terraform in GHA (prefer TF for plan/apply parity)
- Terraform execute(plan) / execute(apply) → Two workflow_dispatch Actions + INTEGRATION_ACTION
- Issues wait(approval) + DevOps sendApprove → Port INPUT node (POC); optional later: GitHub Issue + callback
- waitUntilEnd / sendStatus → reportWorkflowStatus + port-github-action PATCH_RUN / entity upsert
- addWorkflow + createPullRequest + Merge → GHA opens PR; human merge; Port event/status update

8. Demo script

1. Developer opens Create repository in Port and submits the form.
2. Show GitHub Actions plan run and pending approval in Port.
3. DevOps approves → apply creates repo → catalog entity appears.
4. Developer runs Add team, then Configure pipeline.
5. Developer runs Add workflow → PR opened → DevOps merges → Port shows complete.

9. Risks to call out early

- Long-running async state: plan artifacts must be keyed by Port run ID so apply uses the correct plan.
- Two approval styles: Issue-based (UML) vs Port INPUT vs PR merge — align with the customer before building Issue automation.
- Workflows are open beta: prefer Workflows for new multi-step logic; legacy Actions & Automations remain supported if already in use.
- Secrets: never put secret values in workflow run outputs or entity properties.